RESEARCH FUNDAMENTALS – MINOR PROJECT PROPOSAL

Building a GPT-Style Language Model from Scratch

A transparent, tested, and fully documented implementation of the GPT architecture — from byte-pair tokenization to trained text generation.

Bikram Sharma • Pramesh Lamichhane • Swikar Poudel
Gandaki College of Engineering and Science | 07 JUL 2026

AGENDA

What We Will Cover

01 **Literature Review**

02 **Introduction & Objectives**

03 **Tools, Methodology & Design**

04 **Research Plan, Testing & Budget**

05 **Model Results & Outcomes**

06 **Contributions & Future Works**

Why Build a GPT from Scratch?

- Most students only ever see the output side of language models — a chat box.
- GPT-style models predict the next token from all previous tokens (Radford et al., 2019).
- Self-attention (Vaswani et al., 2017) replaced recurrence and made this trainable at scale.
- This project builds every component directly — tokenizer, embeddings, attention, training, generation.

Base Repository

GPT-From-scratch

- config.py — GPT-124M configuration
- model/gpt.py, model/transformer.py
- Training + tutorial notebooks
- Sample corpus: the-verdict.txt

github.com/beeks-code/GPT-From-scratch

Problem Statement

“Many learners can use language-model apps, but few can explain — or implement — how tokenization, attention, training, and generation work together.”

Fragmented Tutorials

Tokenization, attention, and training are usually taught as disconnected pieces.

Research Code ≠ Engineering

The base repository works, but lacks tests, configuration logging, and design documentation.

The Gap

No clear, end-to-end, student-built and tested GPT pipeline with proper software documentation.

Project Objectives

1. Study the Transformer/GPT architecture from foundational papers to nanoGPT.
2. Implement embeddings, masked multi-head attention, FFN, LayerNorm, and output head in PyTorch.
3. Build a GPT-2 BPE tokenized dataset pipeline with context windows.
4. Train & validate on a chosen corpus, logging loss, perplexity, and samples.
5. Implement temperature and top-k decoding strategies.
6. Refactor the repository into a clean, tested, reproducible codebase.
7. Produce a full set of UML/ERD/sequence/class design diagrams.

Literature Review & Gap Analysis

From Vaswani et al.'s original Transformer to nanoGPT-style educational rebuilds, the architecture is well published — but rarely reproduced end-to-end, tested, and documented by the students studying it.

Foundational Papers

Vaswani et al. (2017) introduced self-attention; Radford et al. (2019) scaled it into GPT-2.

Educational Rebuilds

Public rebuilds like nanoGPT share training code, but usually skip tests and design docs.

The Gap

No available example pairs the taught theory with tested code and full documentation.

Tools & Technology Stack

Python 3.10+

Core implementation language

PyTorch

Tensors, autograd, training

tiktoken

GPT-2 byte-pair tokenizer

Jupyter Notebook

Experiments & demos

pytest

Automated testing

Git & GitHub

Version control

VS Code

Development environment

CUDA GPU / CPU

Training compute

System Architecture

- Accepts raw input text and converts it into tokens.
- Generates token and positional embeddings for contextual representation.
- Processes embeddings through **12 Transformer layers** with causal self-attention.
- Uses feed-forward networks, layer normalization, and residual connections to learn features.
- Produces output logits to predict the next most probable token.
- Accepts raw input text and converts it into tokens using **Tiktoken BPE**.
- **Dropout** is applied during training to improve generalization and reduce overfitting.
- The final representation is projected to the vocabulary space to generate **output logits**.

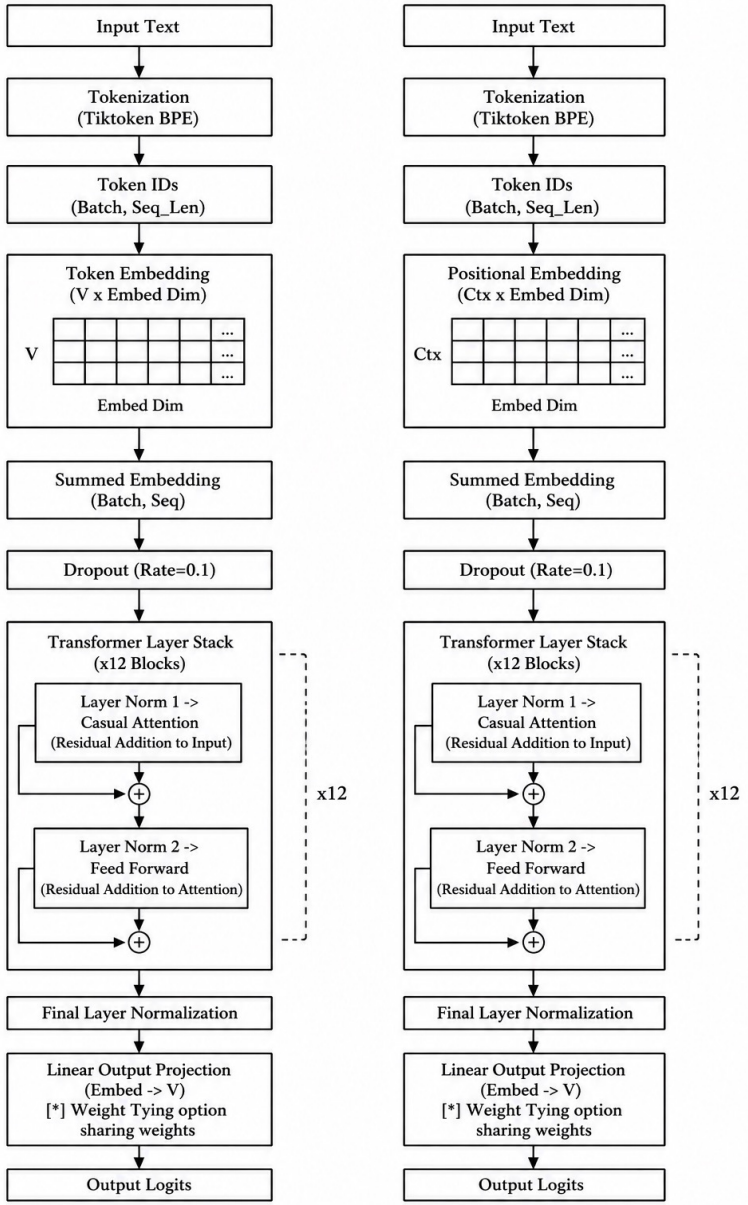


Figure 1: Proposed System Architecture

Use Case Diagram

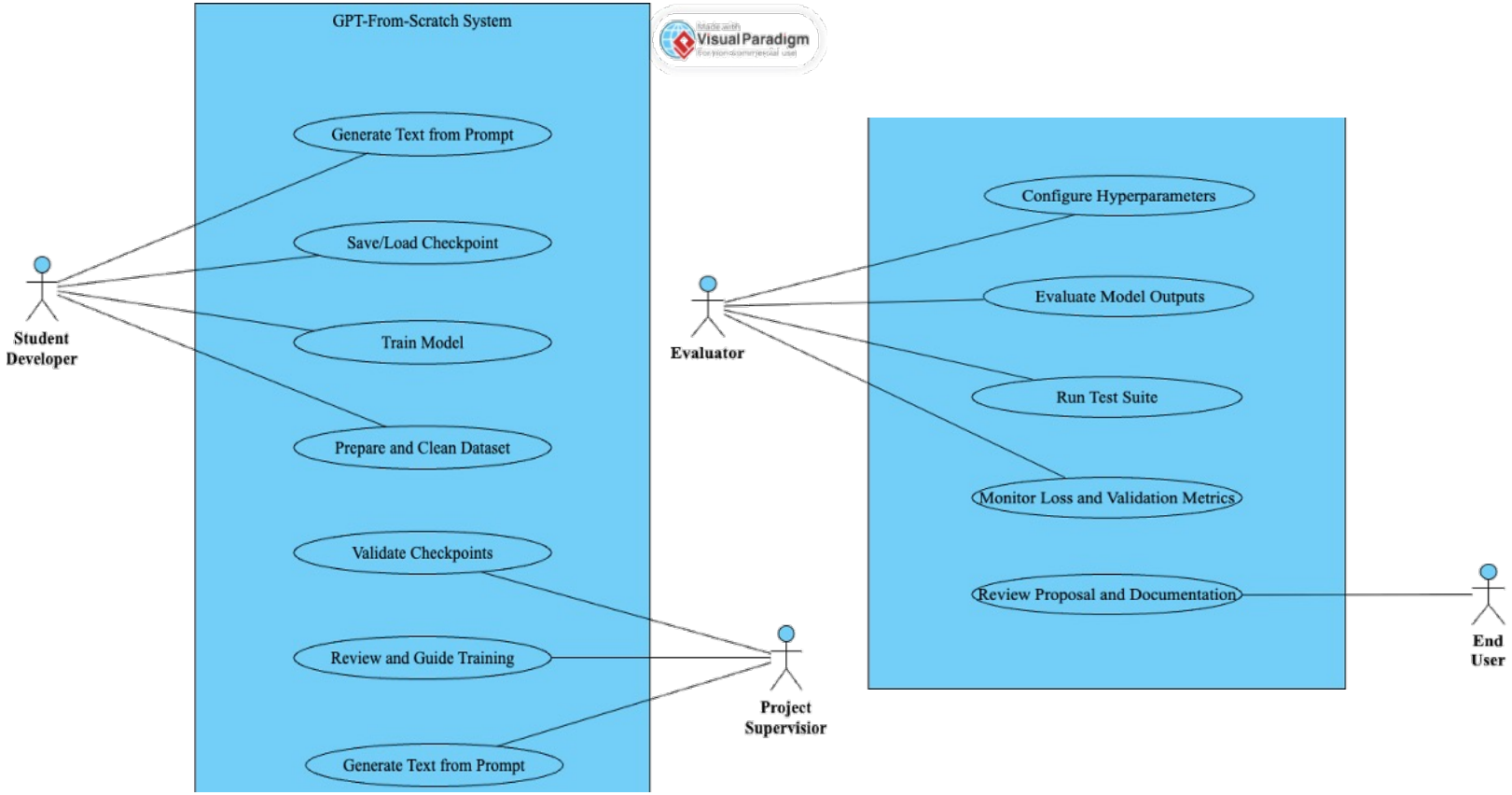


Figure 2: Use Case Diagram

Four actors interact with the system: the student developer, project supervisor, evaluator/panel, and end user.

Nine use cases span dataset prep, training, checkpointing, testing, generation, and documentation review.

ER Diagram – Experiment Tracking

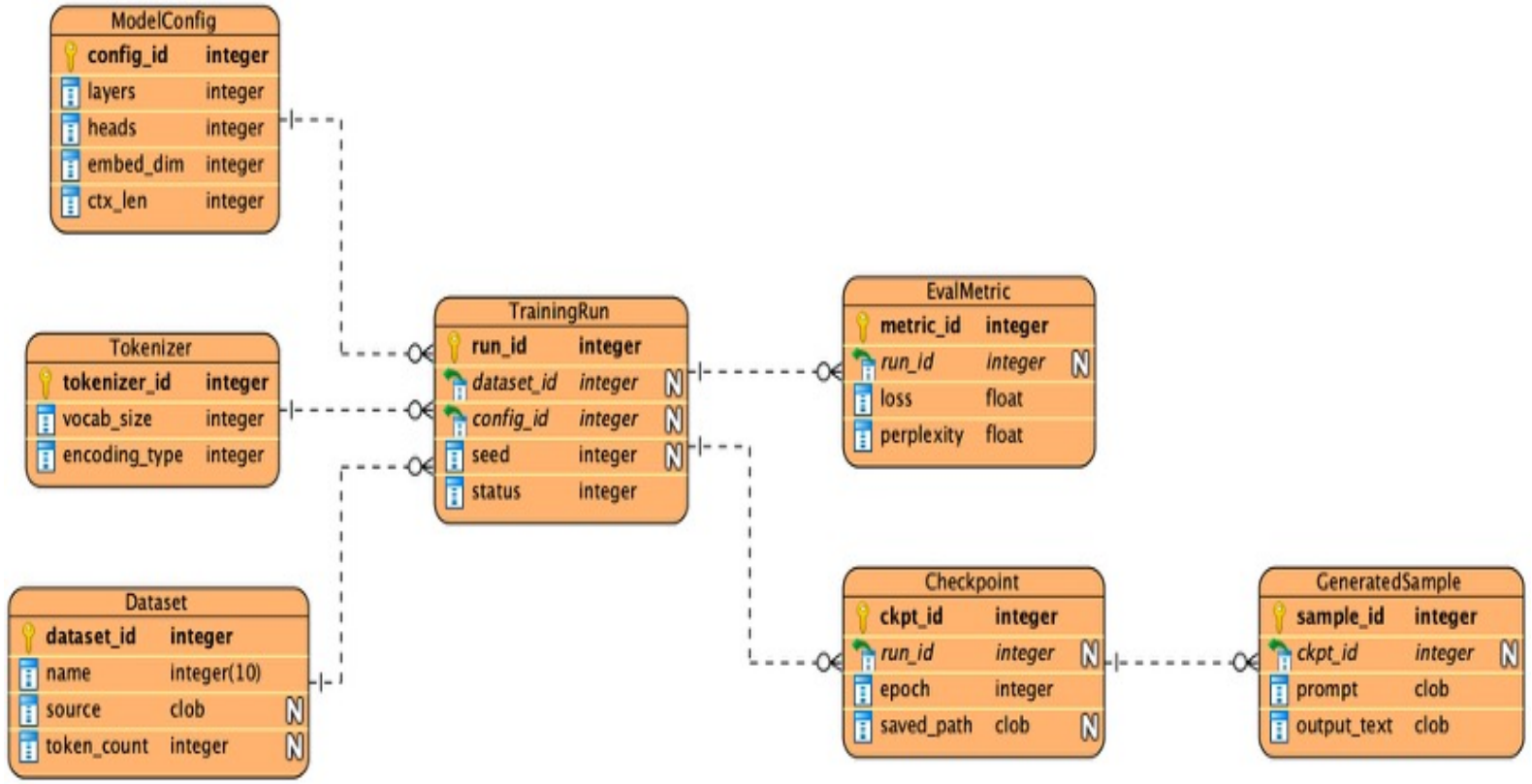


Figure 3: ER Diagram for Experiment Tracking

A lightweight schema (Dataset, Tokenizer, ModelConfig, TrainingRun, Checkpoint, EvaluationMetric, GeneratedSample) makes every result traceable to the exact configuration and data that produced it.

Sequence Diagram – Text Generation

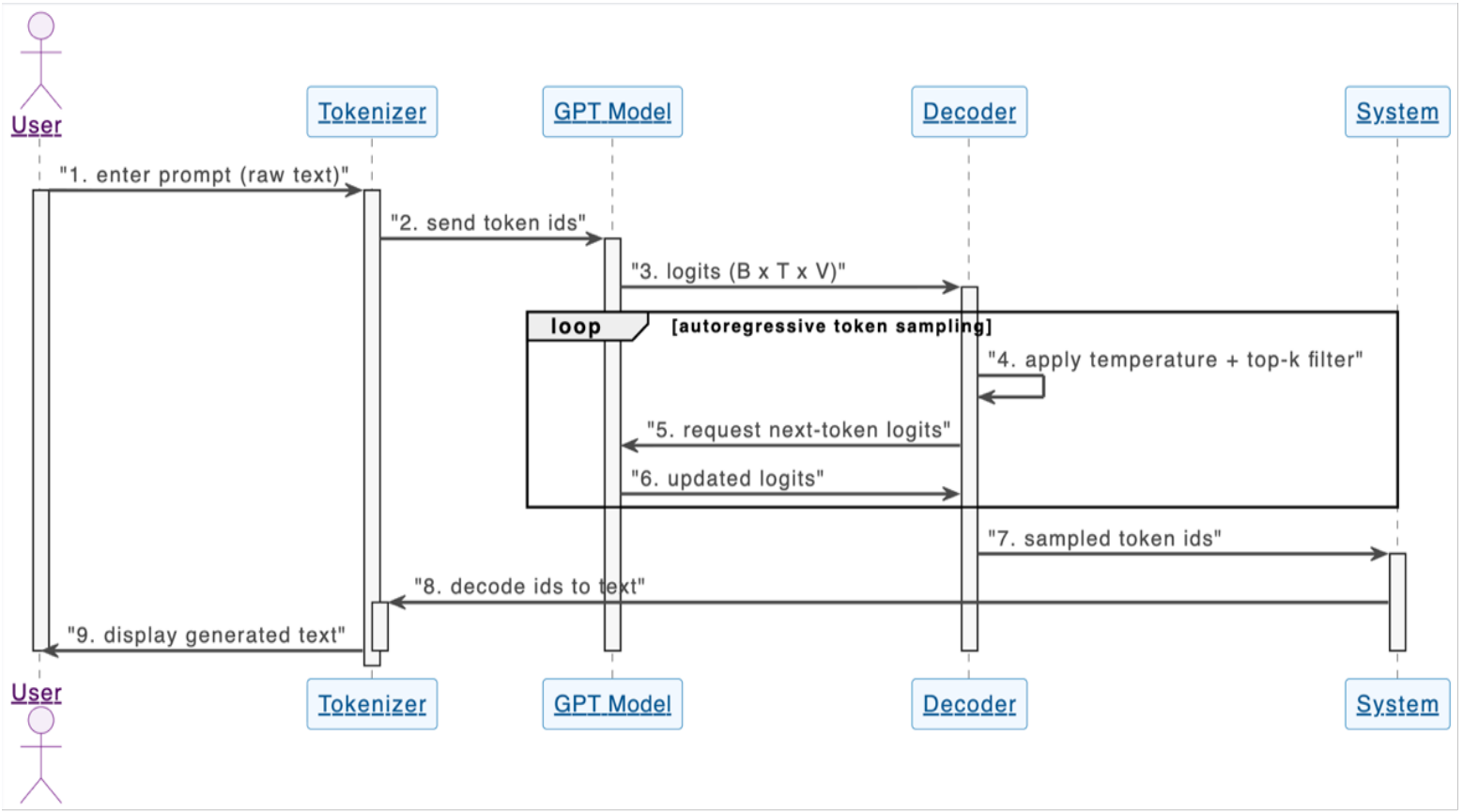


Figure 4: System Sequence Diagram – Text Generation

A prompt is tokenized, passed through the model, filtered by temperature and top-k, sampled autoregressively in a loop, then decoded back into readable text.

Sampling Strategies

Three decoding strategies were implemented and compared for text generation.

Greedy

Always picks the highest-probability token — fully deterministic, no randomness.

Temperature Scaling

Divides logits by temperature before softmax; lower T sharpens, higher T adds randomness.

Top-k Sampling

Keeps only top-k tokens, renormalizes, and samples via a multinomial distribution.

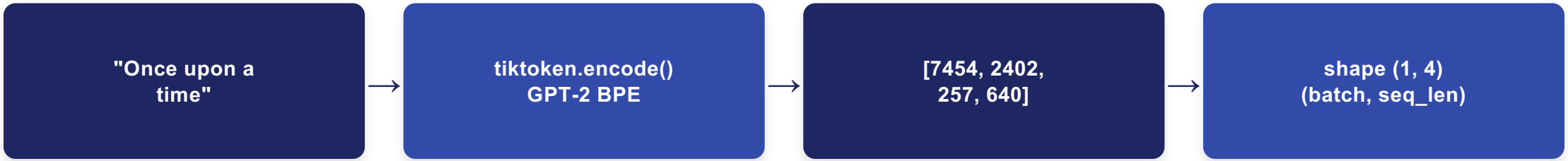
How GPT Generates Text

A step-by-step walkthrough of the complete inference pipeline — from raw text to a sampled next token, built and understood from first principles.

1. **Tokenization — text to numbers**
2. **Token + Positional Embedding — numbers to vectors**
3. **12 Transformer Blocks — where the magic happens**
4. **Layer Norm & Residual Connections — stability at depth**
5. **Project to Vocabulary — hidden states to word scores**
6. **Sampling Next Token — controlled randomness**
7. **Append & Repeat — autoregressive generation**

1. Tokenization – Text to Numbers

Every word or sub-word is converted into an integer token ID using the GPT-2 BPE tokenizer (tiktoken). The vocabulary has 50,257 unique tokens.



Why Byte Pair Encoding (BPE)?

```
tokenizer = tiktoken.get_encoding("gpt2")
Common words -> single token ("the" -> 262)
Rare words -> split into sub-words ("unhappy" -> "un"+"happy")
No unknown words -- byte-level fallback handles any input
```

Vocab size: 50,257 · Max context: 256 tokens · Next: each token ID is looked up in the embedding table

2. Token + Positional Embedding Numbers to Vectors

What is Tokenization?

Token IDs are just integers they carry no meaning. Embeddings convert each ID into a rich 768-dimensional vector where similar words have similar directions in space.

Token Embedding

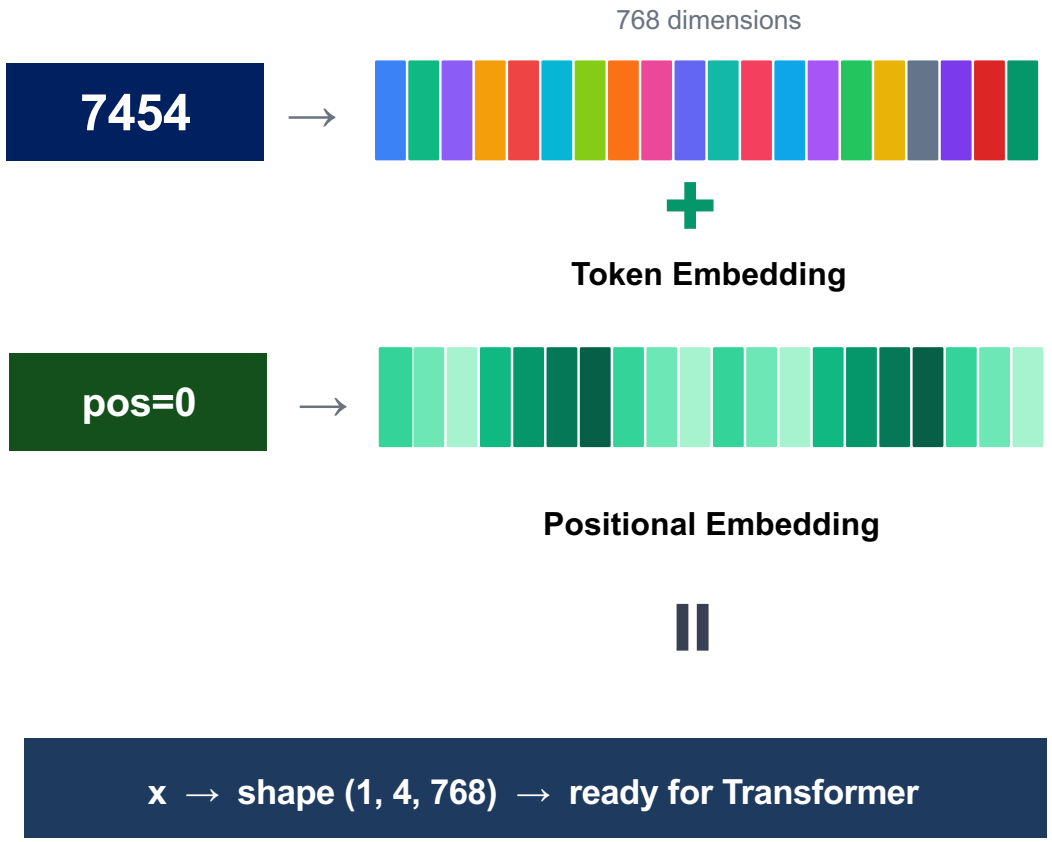
nn.Embedding(50257, 768)
Lookup table: token_id → 768-dim vector
Learned during training — similar words cluster together

Positional Embedding

nn.Embedding(256, 768)
Position index → 768-dim vector
Tells the model WHERE each token is in the sequence

```
x = drop_emb(token_emb(ids) + pos_emb(arange(seq_len)))
```

Output shape: (batch=1, seq=4, emb=768)



x → shape (1, 4, 768) → ready for Transformer

3. 12 Transformer Blocks – Where the Magic Happens

Inside One Block

LayerNorm
normalize activations

MultiHead Attention
12 heads, causal mask, d_k-64

+Residual
add original input back

LayerNorm
normalize again

FeedForward (GELU)
768 → 3072 → 768

+Residual
add to skip connection

What Multi-Head Attention Does

Masked multi-head attention allows a model to look at a sequence from multiple analytical perspectives simultaneously while using a causal mask to strictly prevent it from "cheating" by looking at future tokens.

$$\text{MaskedMultiHead}(\boldsymbol{Q},\boldsymbol{K},\boldsymbol{V}) = Concat\,(head_1, head_1, \dots\dots, head_h)\boldsymbol{W}^o$$

$$\text{Where head}_i = Attention\,(\boldsymbol{QW_i^Q}, \boldsymbol{KW_i^K}, \boldsymbol{VW_i^V}, \boldsymbol{M})$$

$$\text{Attention}(Q', K', V', M) = \text{softmax}\left(\frac{Q'K'^T}{\sqrt{d_k}} + M\right)V'$$

4. Layer Norm & Residual Connections

Layer Normalization

After each sub-layer, activations are normalized to have mean=0 and variance =1 across the embedding dimension. This prevents very large or very small values from destabilising training.

$$norm = (x - \mu)/\sqrt{(\sigma^2 + \varepsilon)}$$

Output = scale × norm + Shift (both learned per layer)

Residual (Skip) Connections

Instead of replacing the input, each sub-layer adds its output back to the original. This creates a "highway" for gradients to flow directly to early layers.

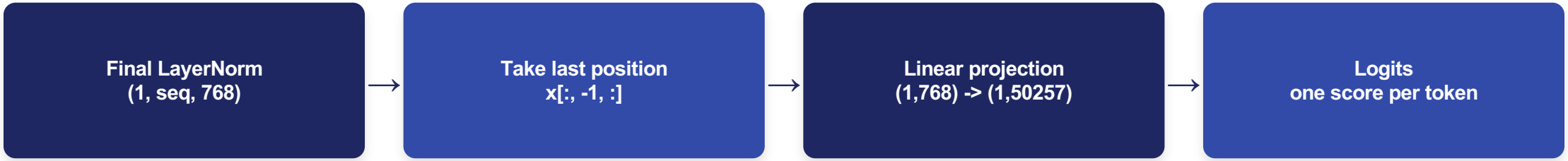
Why residuals? Without them, gradients vanish in deep networks. With them, training a 12-layer model is as stable as training a 2-layer one.

Why we need it:

- **Without LayerNorm** — activations explode or vanish in deep networks
- **With LayerNorm** — training is stable even with 12 stacked layers
- **GPT uses Pre-Norm** — applied BEFORE attention, not after

5. Project to Vocabulary

After 12 transformer blocks, we have a 768-dim vector per position. Only the LAST position matters — it is the model's prediction of what comes next.



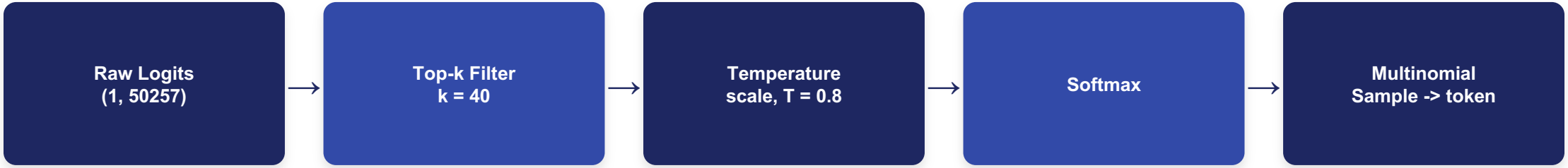
The output head

```
self.output_head = nn.Linear(768, 50257, bias=False)
Higher logit = model thinks that token is more likely next. No softmax here yet.
```

50,257 raw scores (logits), one per vocabulary token — softmax is applied separately during sampling

6. Sampling Next Token – Controlled Randomness

Three steps transform raw logits into the next token, trading off creativity vs. coherence.



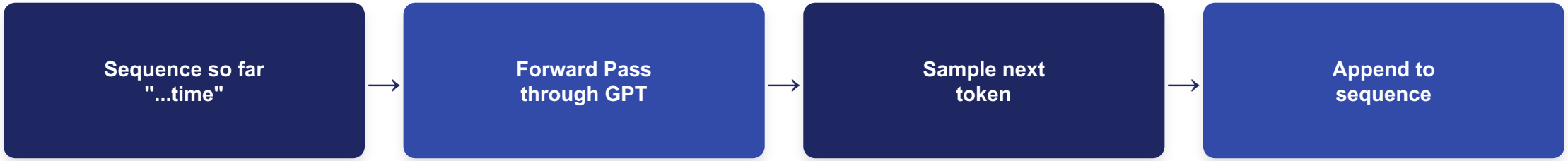
Top-k + temperature + multinomial sampling

```
top_vals, _ = torch.topk(logits, top_k)
logits[logits < top_vals[:, -1]] = float('-inf')
probs = softmax(logits / temperature)
next_token = torch.multinomial(probs, num_samples=1)
```

Unlike argmax, sampling from a distribution introduces controlled creativity — best results at T=0.8, top_k=40

7. Append & Repeat – Autoregressive Generation

Each new token is appended to the sequence and the whole forward pass runs again — one token at a time, each conditioned on all previous tokens.



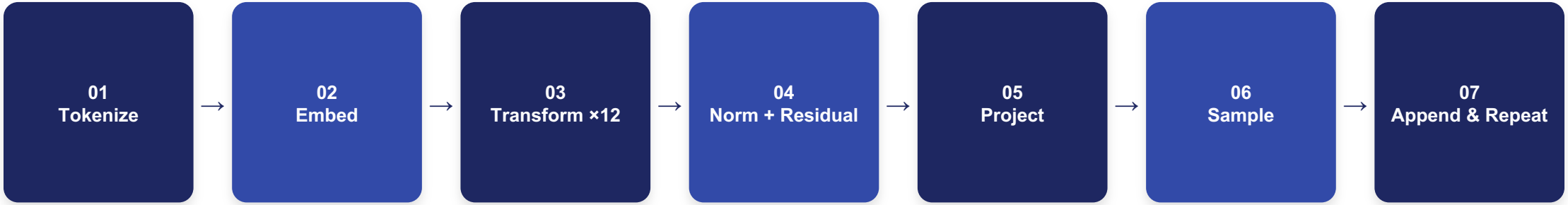
Generation loop (repeats until max_token = 100)

```
for _ in range(max_token):
    ids_cl = ids[:, -256:]            # crop to context
    logits = model(ids_cl)[:, -1, :] # forward pass
    ids = cat([ids, sample(logits)]) # sample & append
```

Cost: generating 100 tokens = 100 complete forward passes through all 12 transformer blocks

The Complete Interference Pipeline

"Once upon a time" → GPT-162M → a coherent generated story



Text -> token IDs (tiktoken BPE) -> 768-dim vectors + position -> 12x attention + feed-forward -> stability via norm/residual -> (1,768)->(1,50257) logits -> top-k / temperature / multinomial -> one new token per pass, repeated up to 100 times.

Sequence Diagram – Training Iteration

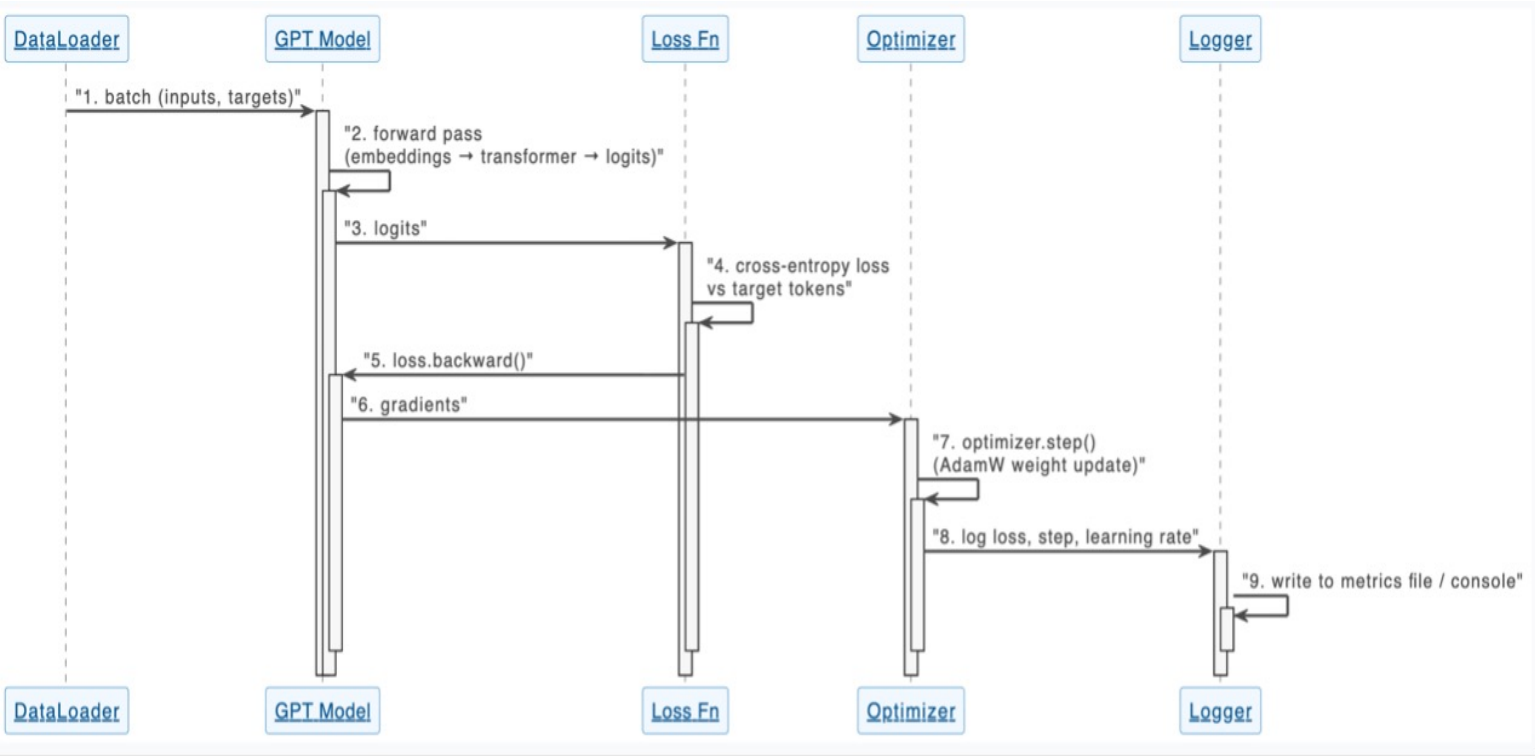


Figure 5: Sequence Diagram – One Training Iteration

One training step: batch → forward pass → cross-entropy loss → backward pass → AdamW update → metric logging.

Design Class Diagram

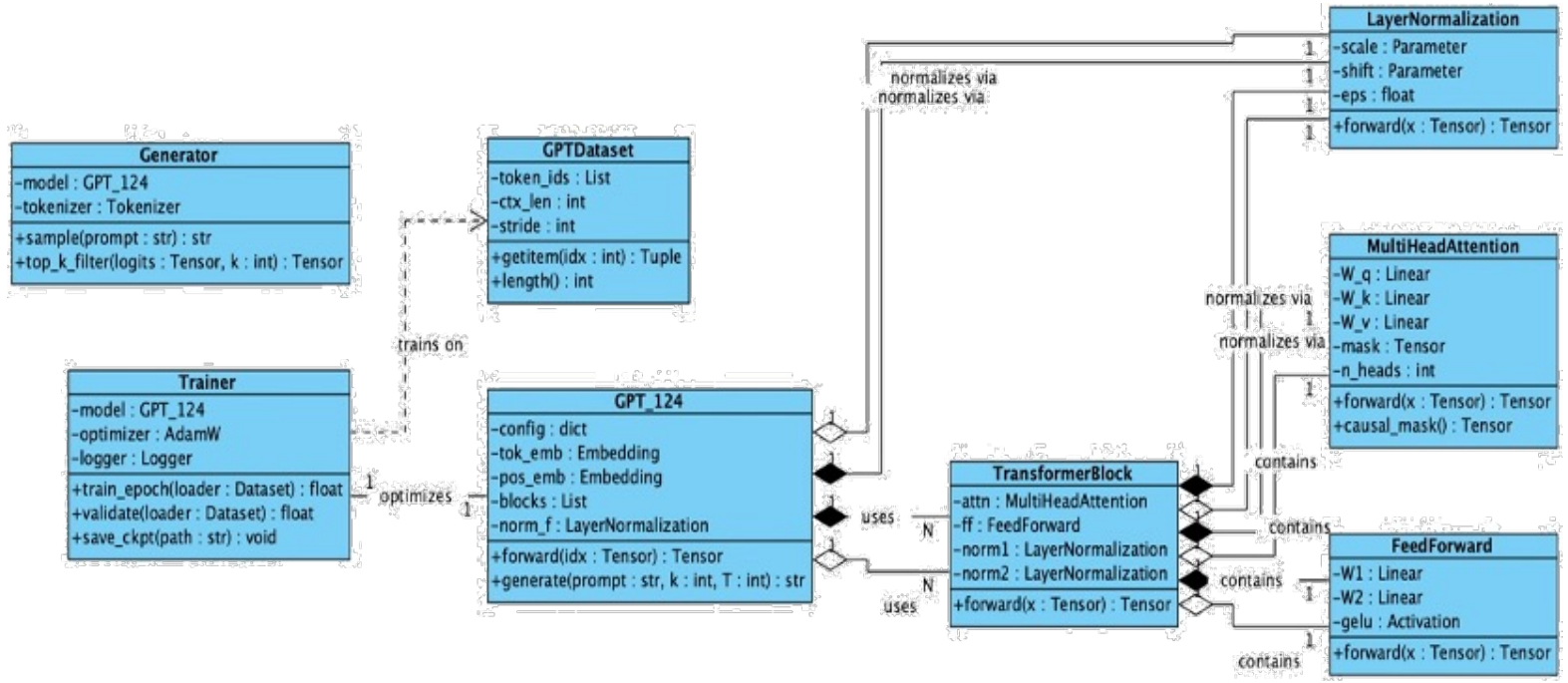
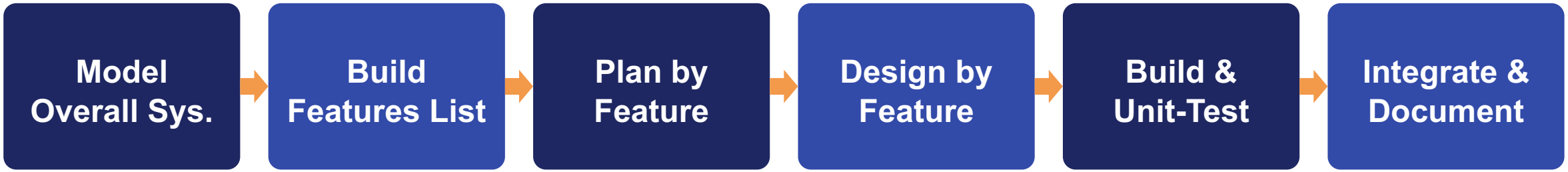


Figure 6: Design Class Diagram

GPT_124 composes a stack of TransformerBlocks, each containing MultiHeadAttention and FeedForward, normalized via LayerNormalization. Trainer and Generator wrap the training and inference workflows.

Feature Driven, Iterative Development



Each feature – tokenizer dataset, transformer block, training loop, evaluation, generation – is designed, implemented, unit-tested, and only then integrated. This keeps debugging local and produces a demonstrable increment every week.

- **Tokenizer pipeline**
- **Dataset pipeline**
- **Transformer block**
- **GPT model**
- **Training loop**
- **Evaluation**
- **Generation**
- **Testing & docs**

Project Structure

A modular layout separates architecture, data, training, and inference into independent pieces.

Core Modules

model/ for architecture, data/ for the dataset and loader, configs/ for hyperparameters.

Pipeline Modules

training/ for the training loop and losses, inference/ for text generation and sampling.

Support

utils/ for metrics and plotting, plus a one-command demo script for reproducible runs.

Team Roles & Data Methods

Responsibility rotates by feature, not by person — every member can read, run, and explain any layer of the system, from tokenizer to decoding.

Team Roles

Shared-ownership model: whoever picks up a feature owns its code, tests, and docs.

Data Collection

Small, clearly licensed or public-domain corpora, starting with the repository's sample text.

Data Analysis

Loss & perplexity tracked every eval interval; samples reviewed alongside the metrics.

Ethical & Responsible-Use Considerations

Even a small, educational language model raises questions worth addressing explicitly, rather than leaving implicit.

Corpus Licensing

Only small, clearly licensed or public-domain text is used — no scraped or unlicensed corpora.

Honest Reporting

Outputs are shown as a demonstration of mechanism, not as factually reliable text.

Known Limitations

Repetition and unreliability are documented alongside results, not hidden behind cherry-picks.

Testing Plan

Test Case	What It Verifies
Tokenizer Test	Encode/decode round-trip is correct
Dataset Shape Test	Tensor shapes match batch × context length
Attention Mask Test	No position attends to the future
Forward Pass Test	Output shape = batch × seq × vocab
Loss Test	Loss is finite, decreases on overfit sample
Generation Test	Output respects length & sampling limits
Checkpoint Test	Reloaded model reproduces identical logits

8-Week Work Schedule

Tasks:

1. Requirement study and repository review
2. Literature review and dataset/tokenizer study
3. Dataset pipeline and data loader refactor
4. Model Implementation review and shape tests
5. Training loop, checkpointing, logging
6. Evaluation, decoding experiments, risk review
7. Unit testing and integration testing and Final report, presentation and demo preparation

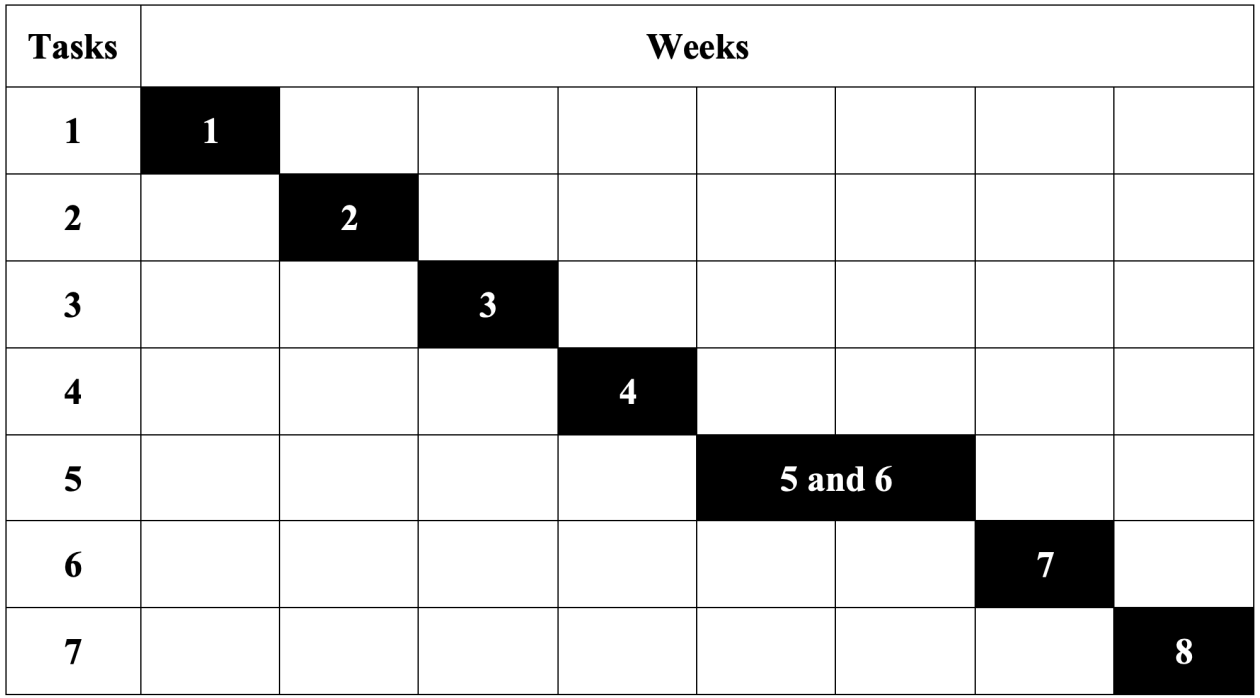


Figure 7: Project Gannt Chart

Risk Matrix & Budget

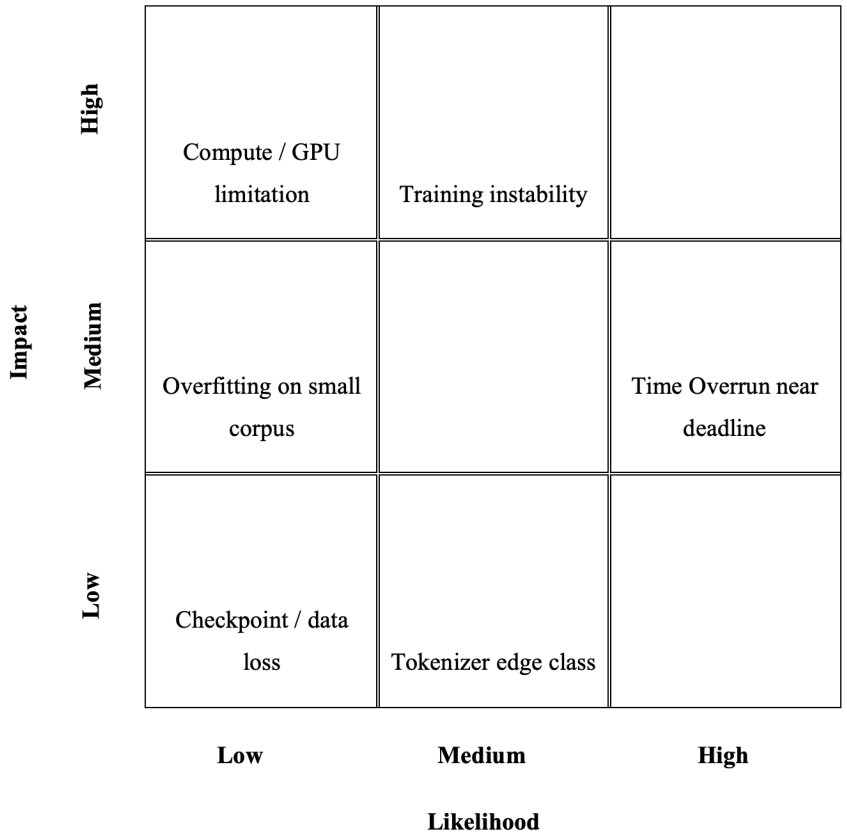


Figure 8: Risk Matrix

GPU/compute limits	Gradient accumulation, cloud fallback
Training instability	Grad clipping, LR warmup, frequent checkpoints
Overfitting	Validation split, dropout, monitoring
Time overrun	Weekly schedule, continuous documentation

Budget: No dedicated budget required — open-source tools, personally-owned hardware, free-tier cloud compute.

Model Training Results

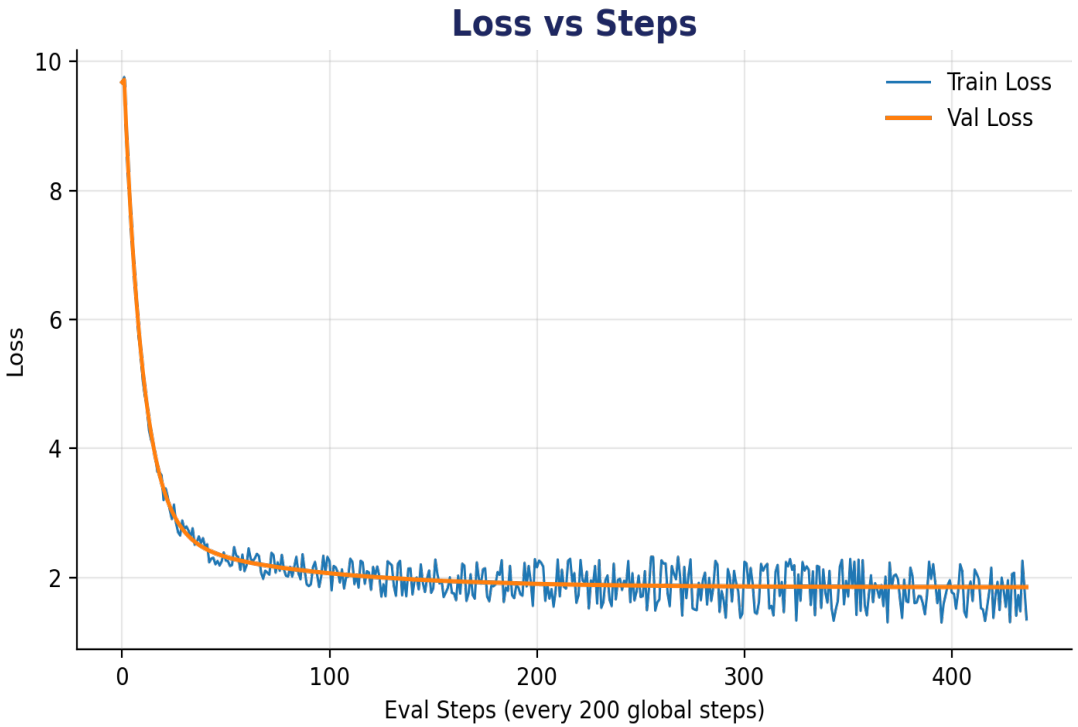


Figure 9: Loss vs Steps

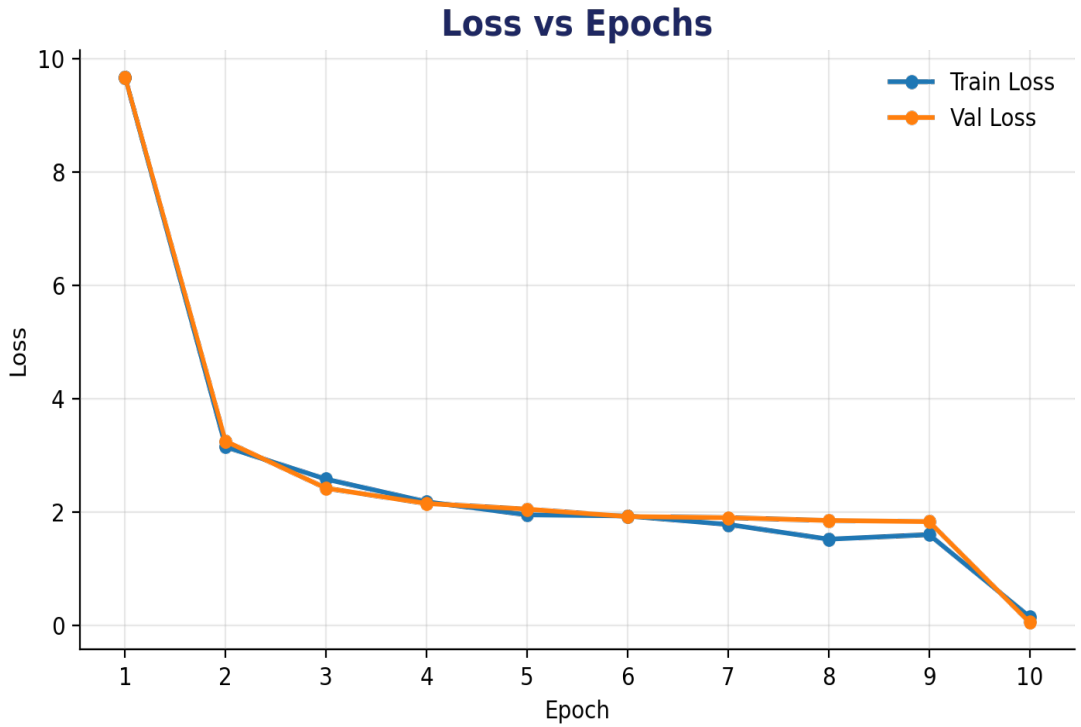


Figure 10: Loss vs Epochs

Generated Text Examples (Illustrative)

Placeholders shown here — replace with real generations once training completes.

Prompt → Output

"Once upon a time" → a short, coherent story continuation in the corpus style.

Sampling Used

temperature = 0.8, top_k = 40 — balances coherence with controlled variation.

Status

Illustrative only — actual samples will be substituted once the model is trained.

Expected Deliverables

- Working, end-to-end GPT-style model trainable on a chosen corpus
- Documented, tested tokenizer & dataset pipeline
- Training/validation logs showing visible learning
- Generated samples under temperature & top-k sampling
- Reproducible repo: config logging, checkpoints, tests, CLI
- Complete proposal, mid-defence report, final report & slides

Contributions

- Modular code structure separating tokenizer, dataset, model, training, and generation
- Automated test suite: shapes, causal masking, loss, checkpointing, generation
- Documented evaluation protocol with tracked loss & perplexity curves
- Complete design artifacts: architecture, use case, ER, sequence & class diagrams
- Explicit feasibility, risk, and ethical-use analysis the original repository lacks
- A disciplined engineering treatment of an existing model, not a new architecture

Future Works

- Cross-lingual extension: repoint the pipeline at a Nepali-language corpus
- Empirical scaling study: vary depth & width, observe loss-vs-parameters
- Lightweight web interface for interactive prompting & decoding comparison
- Instruction tuning on a small instruction-following dataset
- Reuse as a teaching artifact for future Research Fundamentals cohorts
- Continued documentation of model evaluation, bias & hallucination behaviour

Deployment (Planned Extension)

An optional future-work extension — not part of the core proposal's deliverables or claims.

Interface

A lightweight Streamlit app with temperature and top-k sliders for interactive prompting.

Hosting

Free-tier hosting (e.g. HuggingFace Spaces) — no dedicated budget or infrastructure required.

Status

Listed under Future Works (Section 5.6) — not a deliverable of this proposal.

Thank You

SCAN QR

- For Proposal and Presentation

- For GitHub Repository